



Devops

1. Introductie Devops



Flow van een applicatie in kaart brengen flow Hands on: Dev(FI)ops
Reflectie

Levenscyclus van een applicatie

- Bouwen van applicaties is maar een onderdeel van de levenscyclus van een applicatie
- Wat als wij als developer klaar zijn met ontwikkeling?
- Wat als de applicatie draait in productie?
- Wat zit daar nog tussen? Wat komt daarvoor? Wat komt daarna?



De verschillende fases van de software development lifecycle (SDLC)

De termen developer, operations engineer, QA engineer

- Developer
 - Het maken van applicaties in verschillende talen
 - Moet testen voor de applicaties ontwikkelen
- Operations engineer
 - Het beheer, implementatie en onderhoud van IT-infrastructuur
- QA engineer (Quality Assurance Engineer)

- Waarborgen van de kwaliteit van de software

Wall of Confusion

Communicatie en samenwerking problemen tussen development en operations

- Dev
 - Deliver new functionalities (ideally of quality)
 - Wants to deliver
- Ops
 - Guarantee the “run” of applications (stability)
 - Wants to rationalize
- voorbeeld
 - development heeft een nieuwe feature gemaakt maar heeft niet verteld dat systeem vereisten omhoog gaan waardoor het programma crashed

Het verschil tussen functional vs non-functional requirements (lijkt een beetje op userstories)

- Functional requirements (dingen dat zoiezo in het programma moet)
 - **Wat doet het systeem?**
 - Beschrijven wat een systeem moet doen, oftewel de specifieke functies of taken die het systeem moet uitvoeren
 - gericht op functionaliteit en het gedrag van de software
 - Voorbeelden
 - Gebruikers moet kunnen inloggen met een gebruikersnaam en wachtwoord
 - Het systeem moet een rapport genereren op basis van gebruikersgegevens
- Non-functional requirements
 - **Hoe goed doet het systeem dat?**
 - Beschrijven de kwaliteitseisen, prestatiecriteria en beperkingen van een systeem, Ze definiëren hoe het systeem zijn functionaliteit levert

- Gericht op de prestaties, beveiliging, schaalbaarheid en andere kwaliteitsaspecten van de software
- Voorbeelden:
 - De applicatie moet binnen 2 seconden reageren op gebruikersacties
 - Het systeem moet 99,9% uptime garanderen
 - Gegevens moet worden versleuteld tijdens verzending

2. The 3 ways

Agile, DevOps & the 3 ways



Agile VS DevOps

Historiek

The 3 Ways

Reflectie

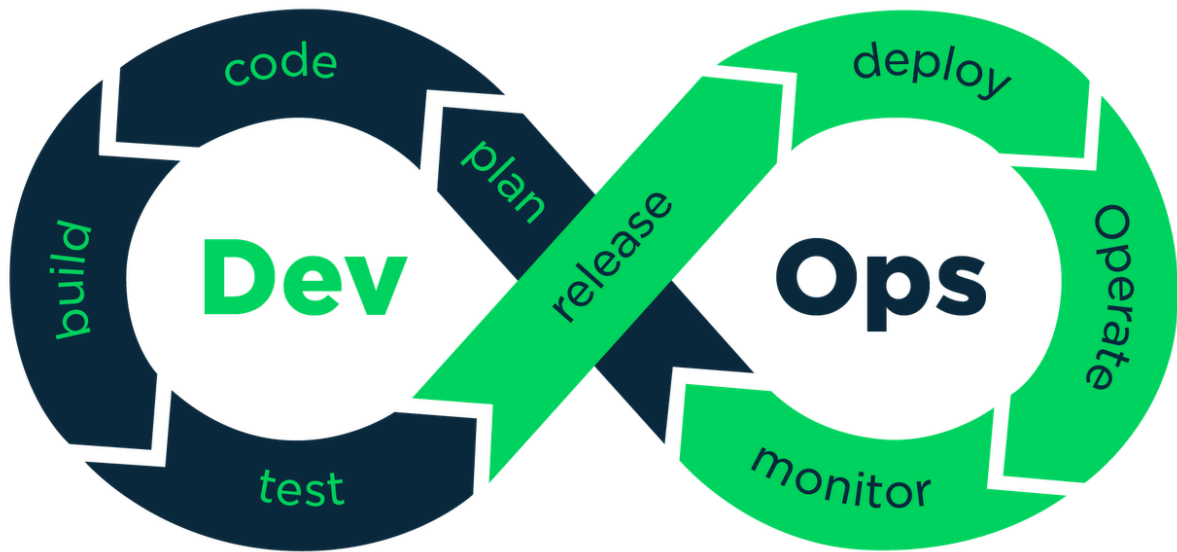
Agile

- Kanban
- Scrum
- Lean
- FDD

Waterfall

- een lineaire softwareontwikkelmethode waarbij elke fase volledig moet worden afgerond voordat de volgende begint
- Het is rigide en minder flexibel voor veranderingen tijdens het proces

DevOps



DEV

- Application Performance
 - Decrease latency by using APM Tools
- End User Analytics
 - Monitor end user latency and check device performance
- Quality code
 - Ensure deployments don't degrade performance
- Code-Level Errors
 - Lower MTTR finding error root causes

OPS

- Application Availability
 - Make sure Uptime and SLA's are in order
- Application Performance
 - Solve problems by correlating infrastructure and application metrics
- End User Complaints
 - Fix problems before end users complain
- Performance Analytics
 - Use automatically generated baselines to focus troubleshooting

Pipeline

- geautomatiseerde workflow die code van ontwikkeling naar productie brengt, bestaande uit stappen zoals bouwen, testen, integreren en implementeren
- Het stroomlijnt processen en vermindert handmatige interventies

Hoe verhogen we de "Flow"

Kleinere stapjes, maar meer releases betekent:

- Minder WiP (Work in progress)
- Makkelijker om fouten te vinden en op te lossen
- Maakt werk zichtbaarder

De Filosofie achter DevOps:

The 3 Ways

The principles of Flow

- Waterfall(van boven naar beneden)
- Agile (van boven naar beneden, in loopjes)
- DevOps (van links naar rechts naar links, altijd in beweging)

Swarming

- Als een medewerker problemen heeft dat niet opgelost geraakt
- Iedereen op 1 probleem laten focussen totdat het probleem opgelost is

Technical debt

- extra inspanning die nodig is om slechte of suboptimale oplossingen in software te verbeteren of te vervangen

Verschillen Agile en devops

- Agile
- DevOps

- Iteratief
 - sprints van 1/4 weken
- customer collaboration
- Maakt de software en geeft het aan de klant
- end-to-end software delivery system
 - (development and deployment)
- Automation
 - CI/CD and monitoring
- Continuous workflow with ongoing integration, delivery and monitoring

devops Ketting

- Plan
- Develop
- build
- test
- release
- deploy
- operate monitor

verschillende focuspunten binnen devops voor developers en operation engineers

- | | |
|--|--|
| <ul style="list-style-type: none"> • Dev <ul style="list-style-type: none"> ◦ Application performance <ul style="list-style-type: none"> ▪ Decrease latency by using APM Tools ◦ End user Analytics <ul style="list-style-type: none"> ▪ Monitor end user latency and check device ◦ Quality Code | <ul style="list-style-type: none"> • Ops <ul style="list-style-type: none"> ◦ Application Availability <ul style="list-style-type: none"> ▪ Make sure Uptime and SLAs are in order ◦ Application Performance <ul style="list-style-type: none"> ▪ Solve problems by correlating infrastructure and application metrics |
|--|--|

- Ensure deployments don't degrade performance
- Code-level Errors
 - Lower MTTR by finding error root causes
- End User Complaints
 - Fix problems before end users complain
- Performance Analytics
 - Use automatically generated baselines to focus troubleshooting

The first way

Identificeer de onderdelen die zorgen dat de business-idee omgezet wordt in een realiteit

- Code, Infrastructuur, Data, Testen, Nieuwe features = Flow

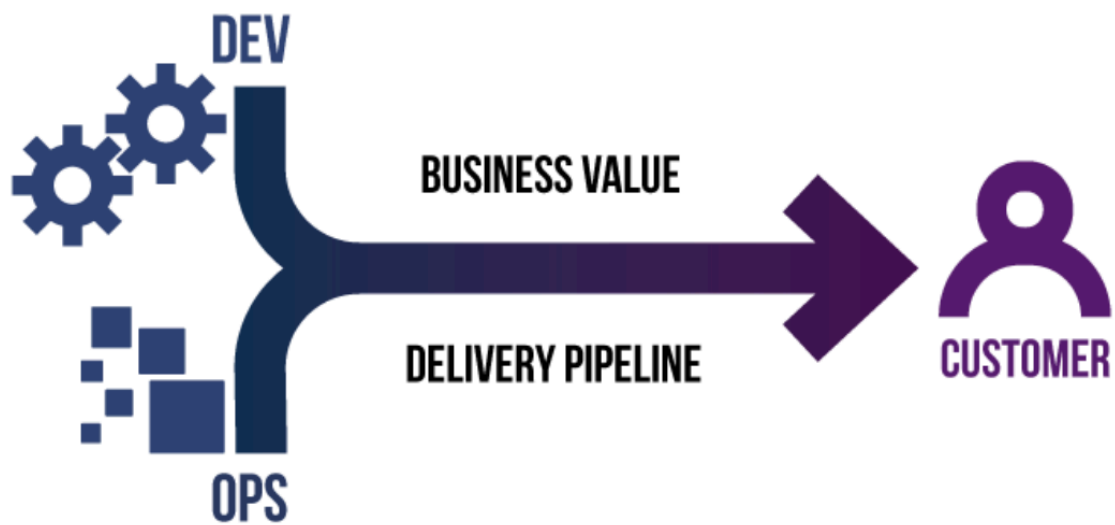
Dev(Business) → Ops(Customer)

Wat is een Pipeline?

Definitie: "deployment pipeline"

It ensures that all code checked in to version control is automatically built and tested in a production-like environment.

- Sleutelwoorden
 - Alle code
 - Versiebeheer
 - Automatisch gebouwd
 - Automatisch getest
 - Productie-waardige omgeving



The Second Way

The principles of feedback

- Waterfall (enkel op het einde)
- Agile (enkel op het einde van de sprint)
- DevOps (continue)

Door continue te monitoren & testen en ervoor te zorgen dat we problemen opmerken als ze gebeuren, ze aan te pakken en op te lossen alvorens ver te gaan



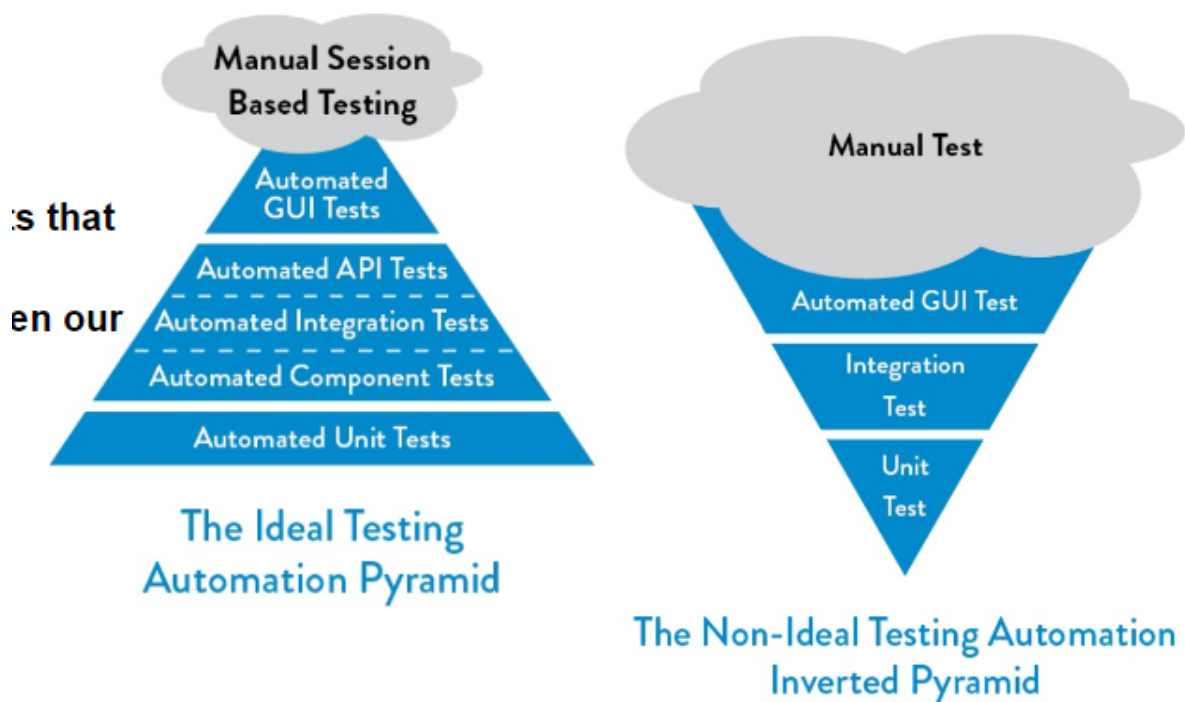
Test ALLES vanaf het moment dat het kan!

Test automatisch om CI te bekommen

Definitie continuous integration:

- A comprehensive and reliable set of automated tests that validate we are in a deployable state.
- A culture that “stops the entire production line” when our validation tests fail. (=digital andon cord)

Ideal vs Non-ideal Testing Pyramids



Problemen oplossen

Swarming

stop de pipeline, gooi het probleem in de groep en los het op.

Swarming zorgt ervoor dat dit snel en efficient kan

Technical Debt

Problemen nu oplossen voorkomt issues later!

Problemen laten liggen" omdat er nu geen tijd is" zorgt voor Technical Debt

THE SECOND WAY

AMPLIFY FEEDBACK LOOPS



The third Way

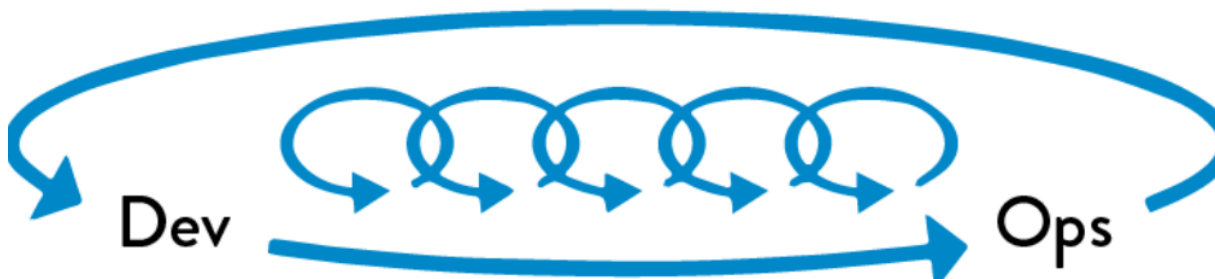
The creation of a generative, high-trust culture that supports a dynamic, disciplined, and scientific approach to experimentation and risk-taking, facilitating the creation of organizational learning, both from our successes and failures

- Enable safety
- Improvement of daily work
- Local discoveries to global improvements
- Inject Resilience

Dit is de moeilijkste, en heeft vaak ook te maken met een cultuur shift binnen bedrijven

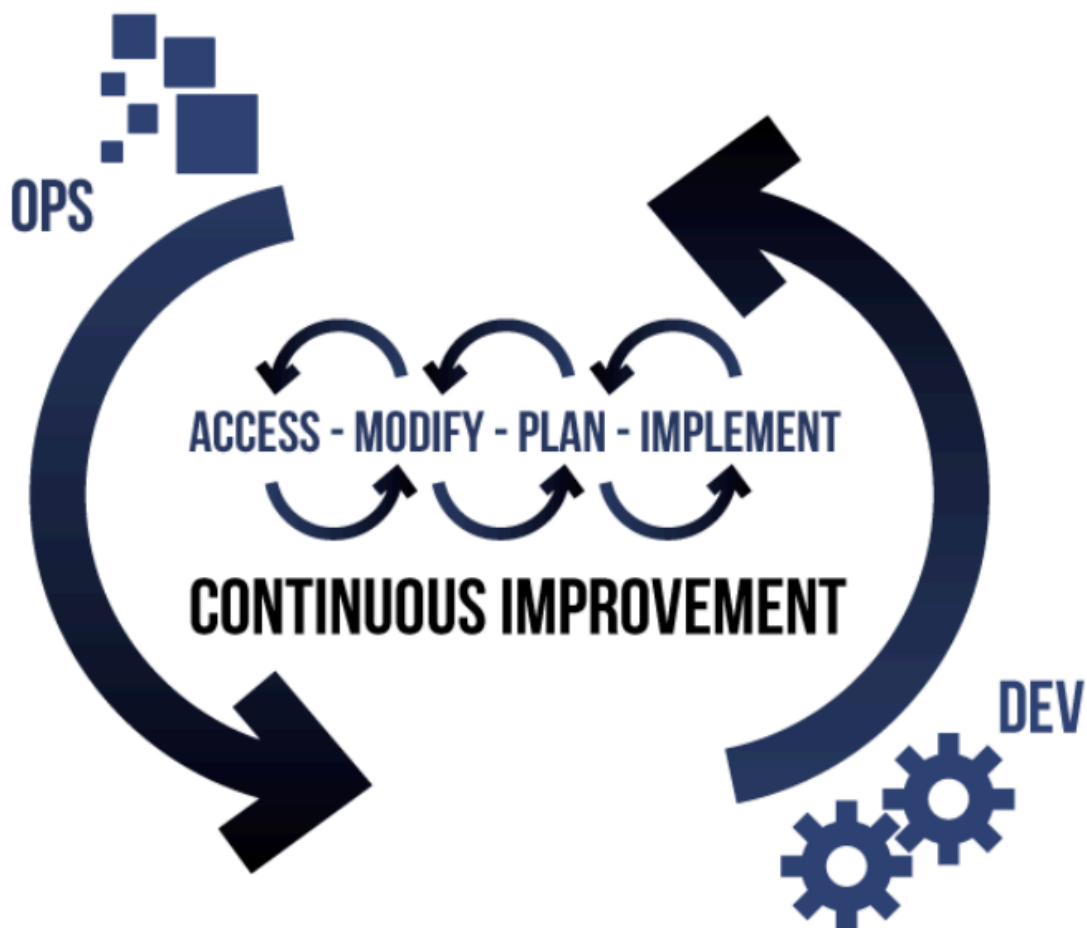
(Business)

(Customer)

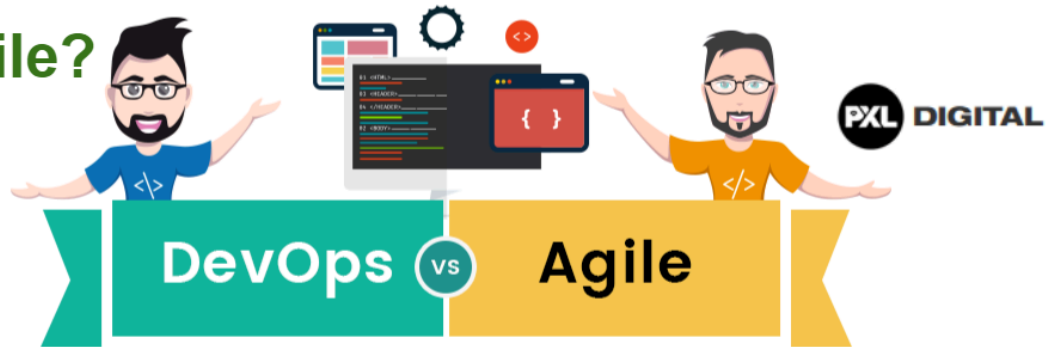


THE THIRD WAY

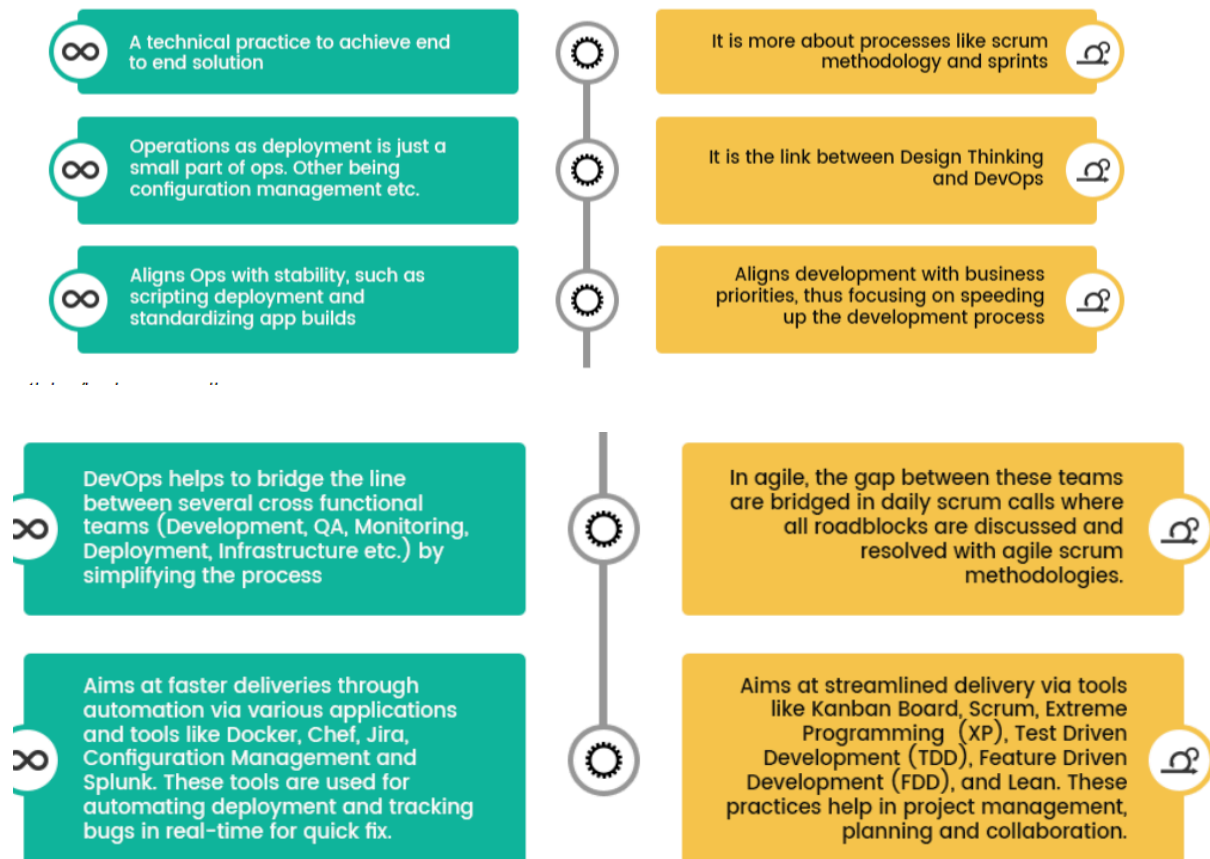
LEARN & EXPERIMENT CONTINUOUSLY



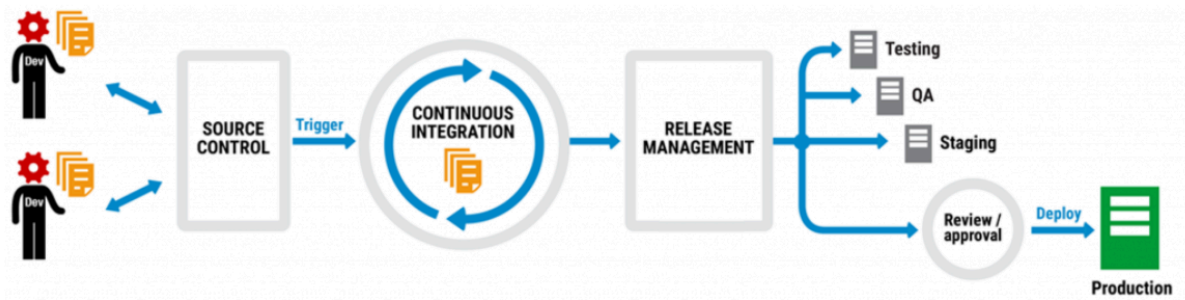
Is Agile?



What's the difference?



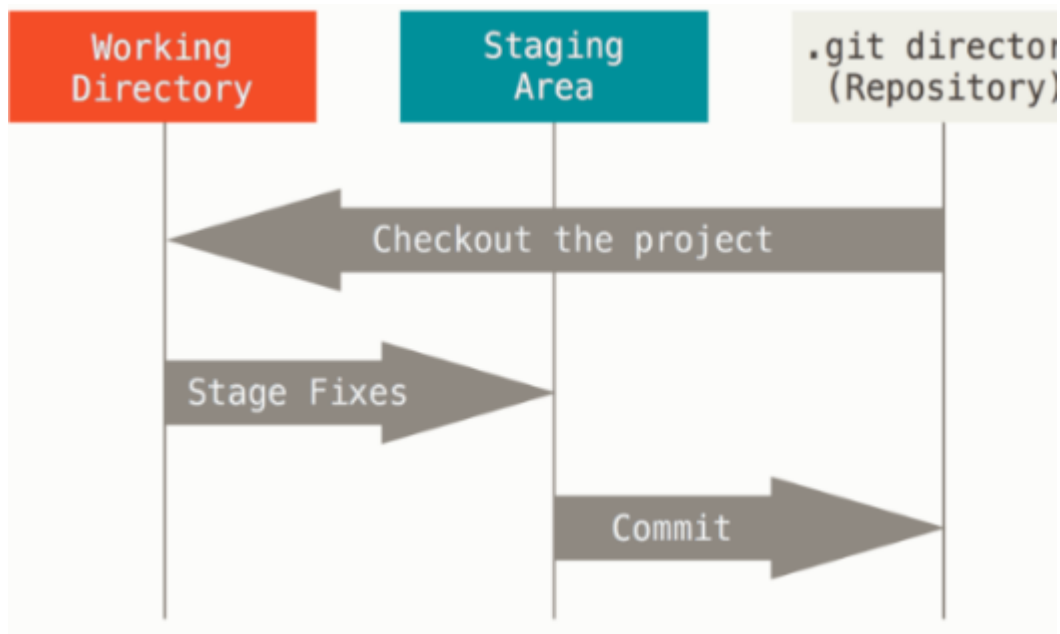
3. Source Control



- Waarom source control
 - Single source of truth (belangrijk binnen flow)
 - Samenwerken als team aan dezelfde code
 - Feedback over nieuwe release van de code

Recap & general use

- Source control
- Waar voorzien:
 - Software code
 - Infrastructure as code
 - (tekst) document
 - ...
- Git, SVN, TFS, ..

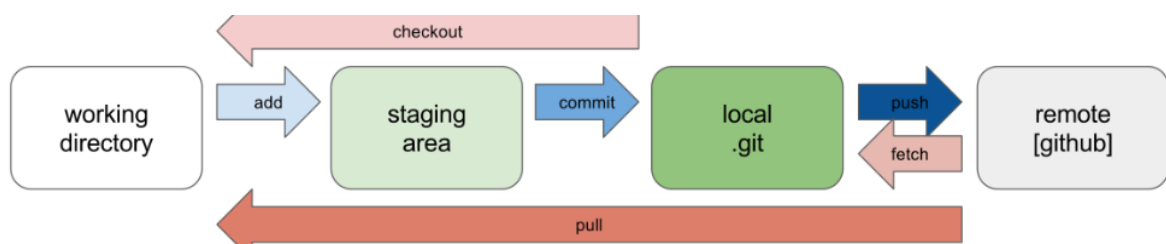


1. Make changes → working
 - a. `git add .`
2. Add changes → Staging index
 - a. `git commit -m ""`
3. Commit changes with a message → repository

Commit message

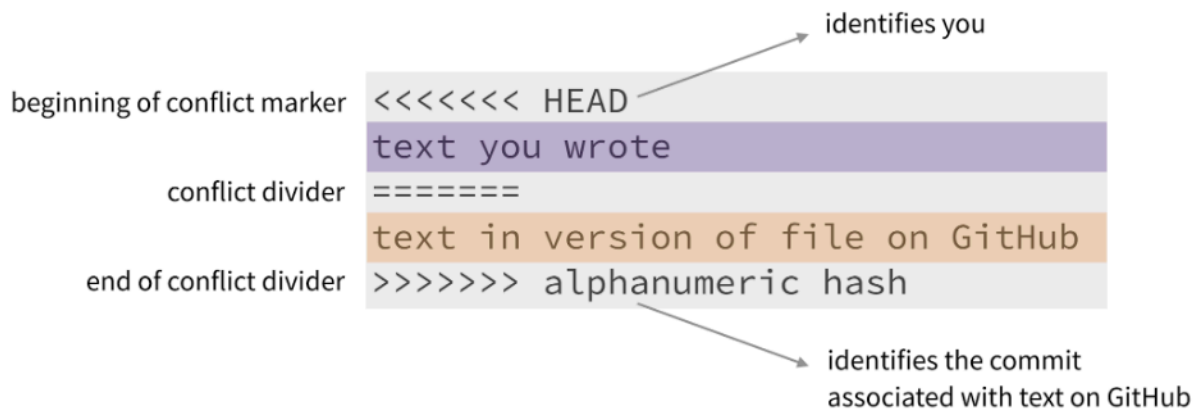
- Geeft feedback over wijzigingen
- Duidelijke flow in aanpassingen code
- Multi line commits

Pushing code to a remote, Central location



Merge conflicts

- Changes on the same line
- Changes in a deleted file

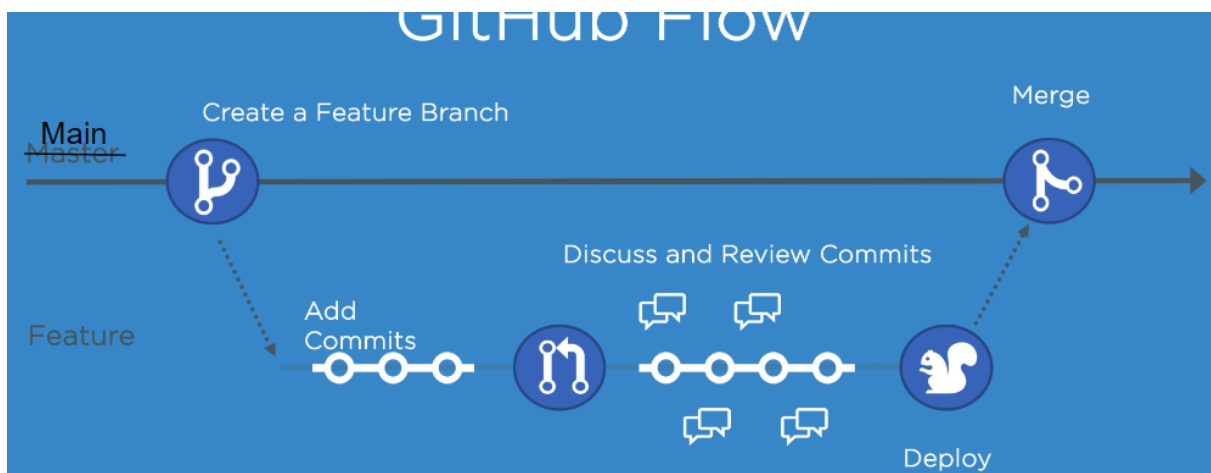


Samenwerken op github repositories

You can go to the settings of a repository and select "Manage access"

Branching strategie

- Strategie voor het (samen)werken met versiebeheer
 - Verschillende varianten
- Aftakkingen van de main branch tijdens het ontwikkelen
- Duidelijke Flow doorheen het ontwikkelproces

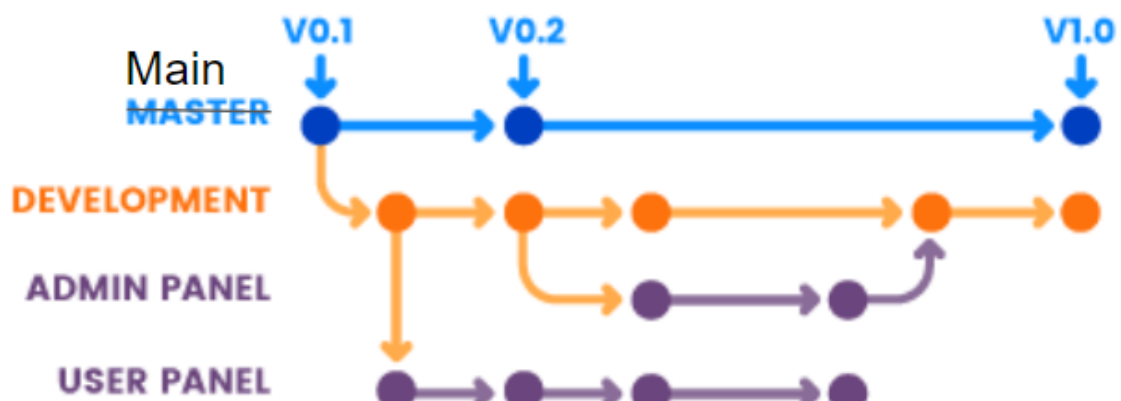


Github flow - Pull request

1. Een developer maakt een feature in een aparte branch in zijn lokale repo
2. De developer pusht de branch naar de centrale repository op Github
3. De developer start een pul request via Github
4. De rest van het team evalueert de code, geeft feedback, past aan, ...
5. De verantwoordelijke van de repository merged de wijzigingen naar de main branch en sluit de pull request

Github flow - Branching

- Branches zijn aftakkingen
 - default branch = main / development branch



- Main branch = releases
- Development branch =

Making branches

- **Branches maken**

```
$ git branch nieuwe-branch
```

Of

```
$ git checkout -b nieuwe-branch
```

Traveling to branches

- Branches gebruiken
 - Bestanden in working dir worden overgeschreven

```
$ git checkout branch-naam
```

branches bekijken

```
$ git branch -a
```

- Branches pushen naar Github

```
$ git push origin nieuwe-branch
```

- Branches pullen van Github

```
$ git fetch -all
```

- Merging a branch

- Make sure you are on the main branch

```
$ git merge nieuwefeature
```

- Deleting branches

```
$ git branch -d nieuwefeature
```

Gebruik pull requests om een merge te doen

Github flow - Pull requests

- Signaal geven dat code klaar is voor merge
- Waarom pull requests?
 - Evaluatie van code voor het mergen
 - Communicatie tussen developers, feedback voorzien
- Inhoud pull request:
 - Comments, omschrijving, tag collega's

The screenshot shows the GitHub interface for the repository 'd-ries / jenkins-test'. The 'Pull requests' tab is selected. A pull request for the branch 'awesome-feature' is shown, which is 1 commit ahead of master. The pull request description is 'update dockerfile with whitespace'. The commit history shows three commits: 'src' (6 months ago), 'Dockerfile' (now), and 'Jenkinsfile' (6 months ago).

File	Commit Message	Time
src	test	6 months ago
Dockerfile	update dockerfile with whitespace	now
Jenkinsfile	test	6 months ago

- Pull request templates

- Maak file pull_request_template.md aan in je root directory van de repository
- Markdown syntax (md)
- Commenting & giving feedback on pull requests
- Merging pull requests
 - approval van meerdere teamleden?
 - Merge conflicts?

waarom kan source control zowel onafhankelijk als binnen een devops context:

1. Source Control als onafhankelijk hulpmiddel
 - a. Individueel gebruik:
 - i. Een enkele ontwikkelaar kan Git gebruiken om wijzigingen in een persoonlijk project bij te houden
 - b. Kleine teams:
 - i. Teams kunnen samen wijzigingen beheren, bugs oplossen en nieuwe functies ontwikkelen met behulp van branches en commits
 - c. Geen Integratie vereist:
 - i. Je kunt source control gebruiken zonder CI/CD pipelines, geautomatiseerde testen of complexe infrastructuur.
2. Source Control binnen een DevOps-content
 - a. CI/CD-pipelines:
 - i. Source control wordt geïntegreerd met tools zoals Jenkins, GitLab CI, Azure DevOps, of Circle. Iedere wijziging in de code triggert automatisch builds, tests en deploys.
 - b. Automatisering:
 - i. Met source control als de kern kan iedere commit automatisch acties starten, zoals unit tests, code linting, en zelfs live deployment
 - c. Samenwerking:

- i. DevOps draait om samenwerking tussen ontwikkeling en operations. Source control faciliteert deze samenwerking door traceerbaarheid en het beheren van branches.
- d. Infrastructuur als Code(IaC): Source control wordt ook gebruikt om configuratiebestanden en infrastructuur (kubernetes manifests) te beheren

Samenvatting

- Onafhankelijk gebruik: Alleen gebruikt voor versiebeheer en samenwerking aan code.
- Devops-context: integraal onderdeel van een geautomatiseerd ecosysteem dat ontwikkelings-, test-, en implementatieprocessen stroomlijnt

Waarvoor wordt source control gebruikt:

Het wordt gebruikt om wijzigingen in code en bestanden te beheren, samenwerken in teams mogelijk te maken, en versiegeschiedenis bij te houden, en fouten eenvoudig terug te draaien

Basis commando's git:

Kijk hierboven

- git checkout -b register-feature
 - brengt je naar de register feature branche
- git log --online
- git push origin register-feature
- git fetch - -all
- git merge nieuwefeature
- git branch -d nieuwefeature
 - delete

Goede commit messages:

- description
 - Uitleg over wat je hebt veranderd en of er een issue is gefixt, ook welke dependencies nodig zijn die je bijgevoegd hebt, je kan ook nog motivatie en context toevoegen in de tekst.

```
# Description
Please include a summary of the change and which issue is fixed. Please also include relevant motivation and context. List any dependencies that are required for this change.

Fixes # (issue)
## Type of change
Please delete options that are not relevant.

- [ ] Bug fix (non-breaking change which fixes an issue)
- [ ] New feature (non-breaking change which adds functionality)
- [ ] Breaking change (fix or feature that would cause existing functionality to not work as expected)

# How Has This Been Tested?
Attach files by dragging & dropping, selecting or pasting them.
```

De flow van het werken met git:

1. **Repository:** begin met een lokale of gekloonde repository (git init of git clone)
2. **Working Directory:** Wijzig bestanden in je project.
3. **Staging Area:** Voeg wijzigingen toe aan de staging area met `git add`.
4. **Commit:** Sla wijzigingen op in de versiegeschiedenis met `git commit`.
5. **Branching:** Werk op aparte branches (`git branch` , `git checkout`) voor nieuwe features of fixes.
6. **Merging:** Combineer wijzigingen van verschillende branches met `git merge`.
7. **Remote Repository:** Synchroniseer lokaal werk met een gedeelde repository (bijv. GitHub) via `git push` en haal updates op via `git pull` of `git fetch`.

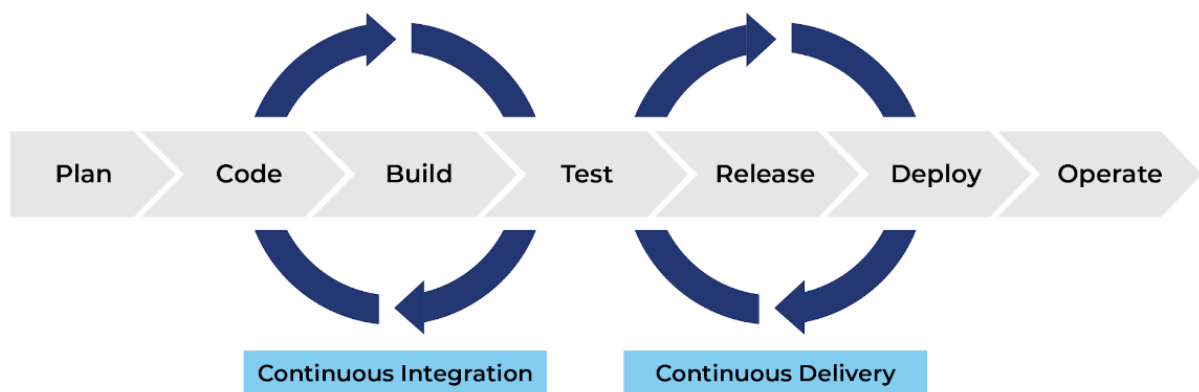
Branching-strategieën

Dit zijn methodes om de structuur van branches in een Github repository te organiseren. Ze bepalen hoe teams samenwerken, nieuwe functies ontwikkelen, bugs oplossen en software releasen. Door een goede strategie te kiezen, blijft de codebase schoon, overzichtelijk en eenvoudig te beheren

H4 Flow - CI/CD & Pipelines

- Definitie "deployment pipeline"
 - It ensures that all code checked in to version control is automatically built and tested in a production-like environment.

CI/CD



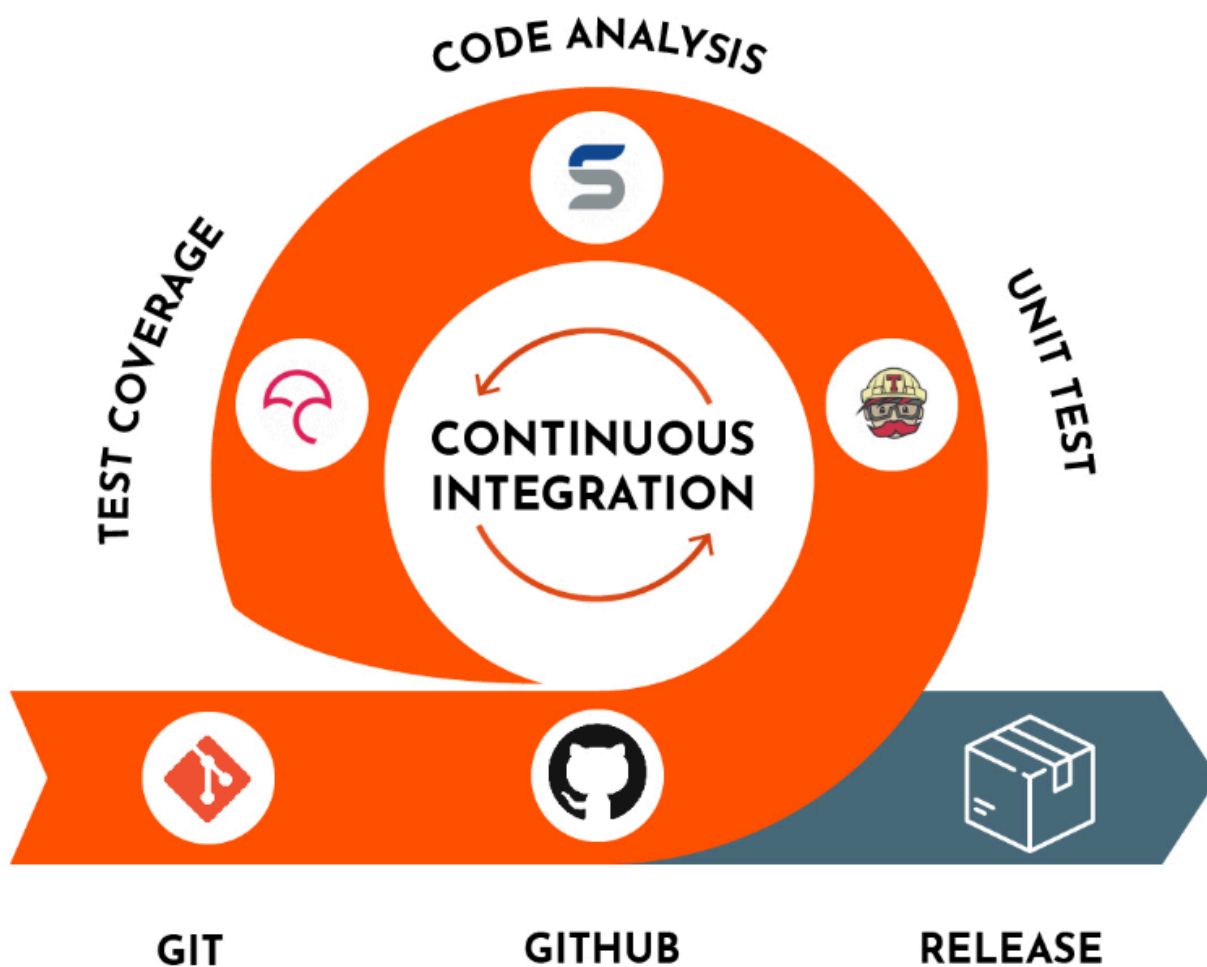
Continuous integration (CD)

- Code binnenhalen
- Dependencies installeren
- Builden / Packagen van de code
- Artifacts / packets beheer

Continuous Deployment/Delivery (CD)

- Continuous Delivery:

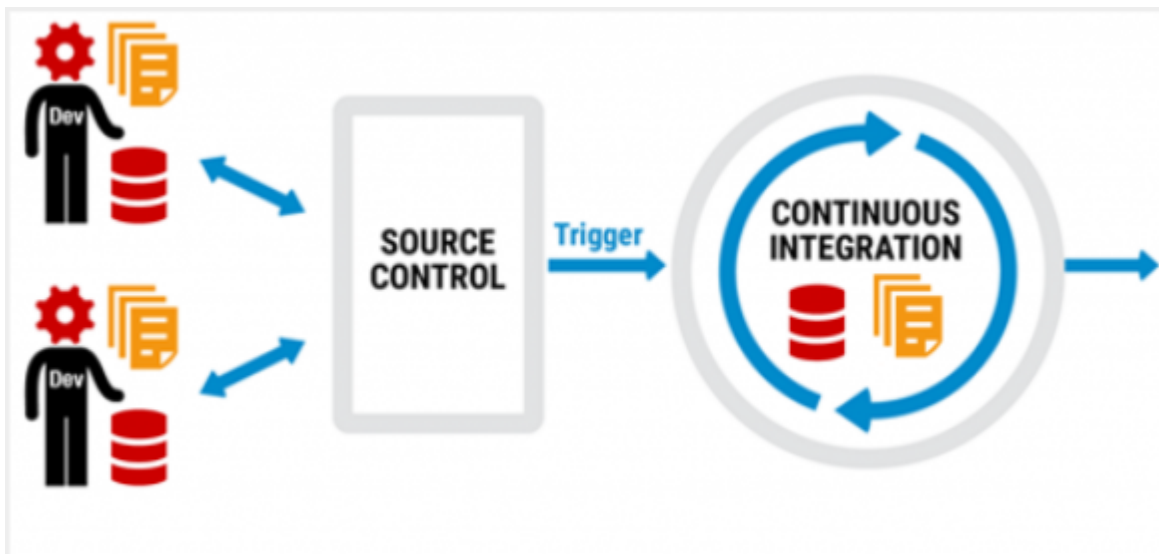
- Automatiseren van het releaseproces, zodat nieuwe versies eenvoudig naar productie kunnen worden gepusht na goedkeuring.
- Continuous Deployment:
 - Automatische release naar productie zonder handmatige goedkeuring, mits alle tests slagen.



Fetching code

- Binnenhalen van de code vanuit source control
- → Brengt deze in de pipeline

Code	Dependencies	build	(unit)test	archive artifact	Deploy to QA	Deploy to production
CI					CD	



Installing dependencies

Bundelen van dependencies in de omgeving

- Afhankelijk van de programmeertaal / omgeving
 - PHP → Composer install
 - Javascript → npm install

Building software

- Automatiseren van het compileren & builden applicatie
 - Afhankelijk van de programmeertaal / omgeving
 - Java → mvn/ant → jar
 - .net core → dotnet publish → exe

Test

- Automatisatie van testen m.b.t code
 - Code quality
 - Linting
 - Unittesten

- ...
- Geen QA omgeving nodig

Archiving artifacts / packaging

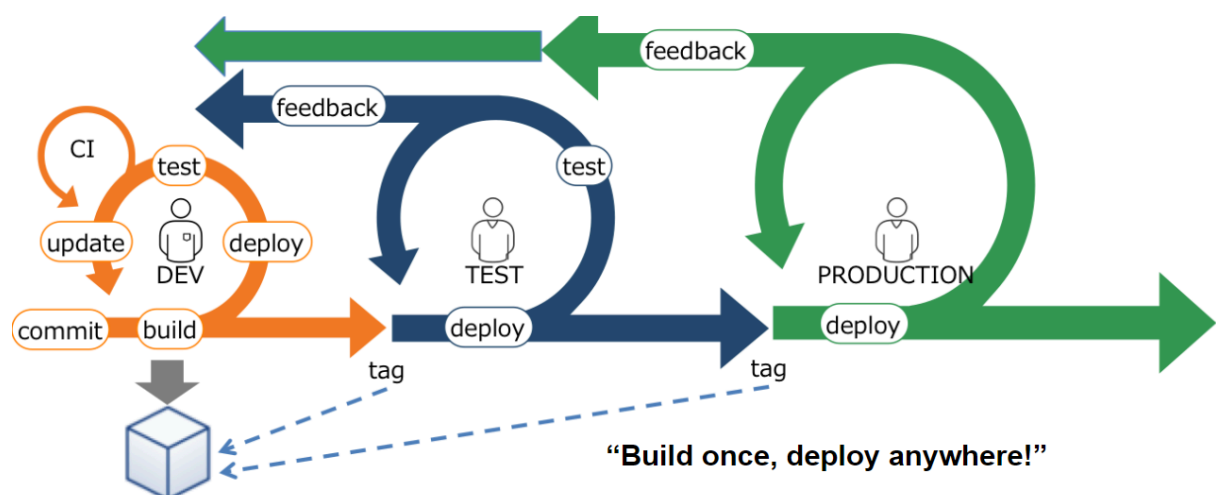
- Beheer van pakketten
 - Centraal? Verschillende versies / Per Build?
 - Build once, deploy anywhere

App	Development	Quality Assurance	User Acceptance	Production
SCM	deployment succeeded	deployment succeeded	deployment failed	deployment succeeded
App	UPack 1.0.7	UPack 1.0.7	UPack 1.0.6	UPack 1.0.4
Sources	UPack 1.0.7	UPack 1.0.7	UPack 1.0.6	UPack 1.0.4

Continuous Delivery / Deployment

CI/CD

A Continuous delivery (CD) pipeline is an automated expression of your process for getting software from version control right through to your users and customers.



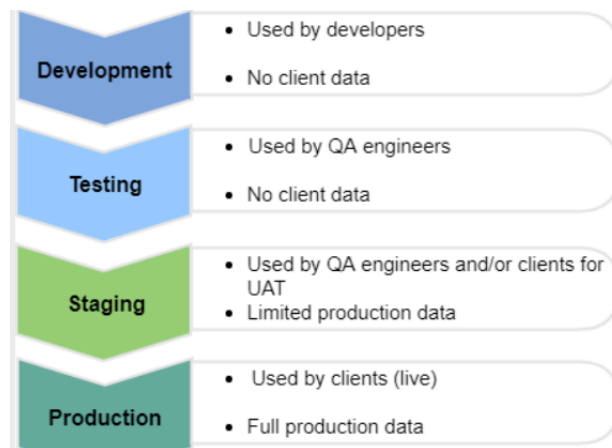
Deploying Software

- Wat is er nodig?

- Gecompileerde code
- Omgeving specifieke parameters
- Dependencies
- Wat nog?

Automatiseren van de deploy

- Vertrekken vanuit de artifacts vanuit onze CI pipeline
 - Centrale repository?
- Omgeving in orde maken
 - Dependencies
 - Omgeving specifieke parameters, credentials, secrets,...



CI/CD pipeline

- Structuur ligt niet vast
 - Elke bedrijf/ omgeving heeft zijn eigen accenten

Maar Waarom

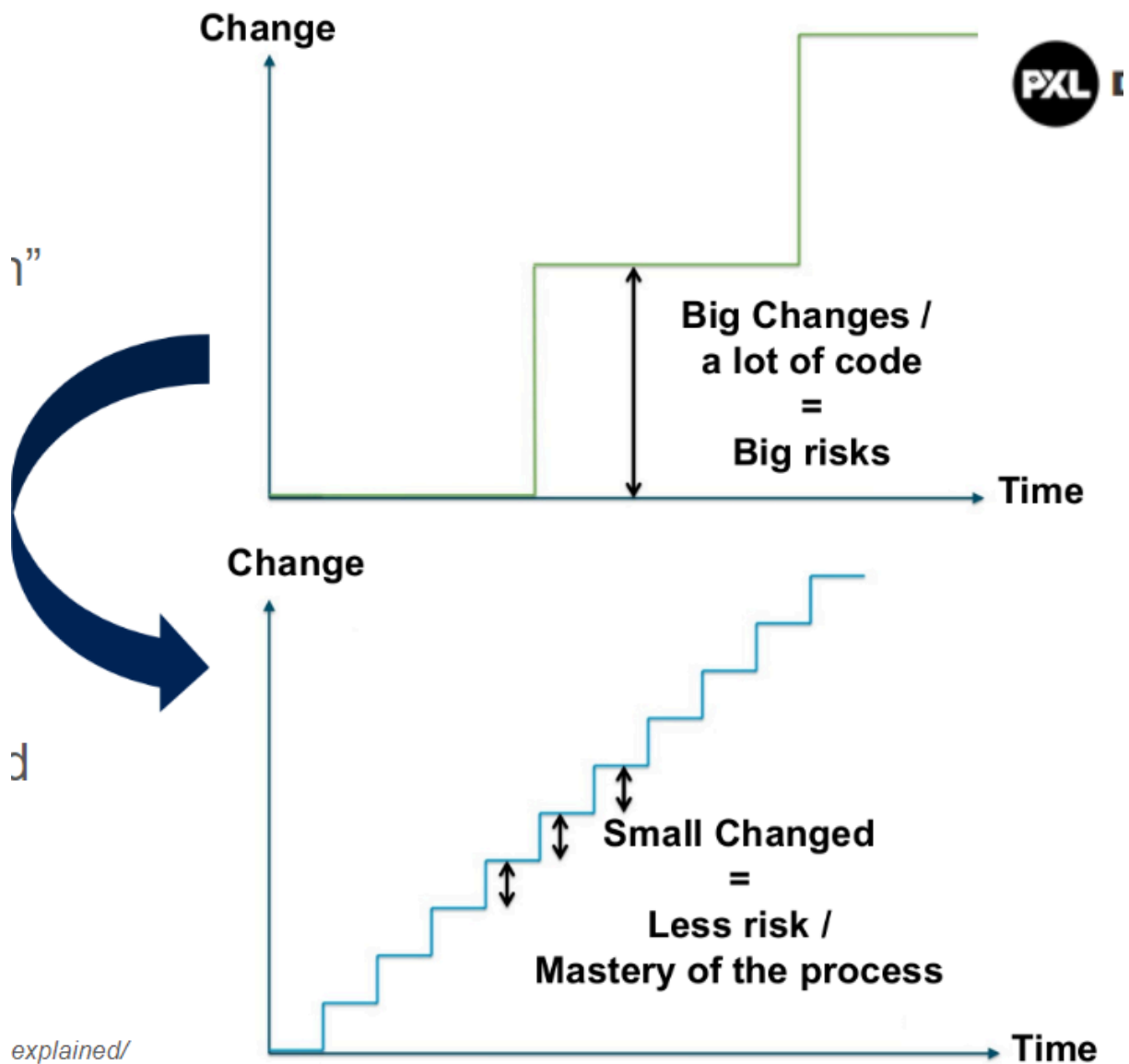
"if it hurts, do it more often"

vergeet apps!

Welkom

- (Micro-)services

- API's
- Front-end & Back-end
- MDM



Jenkins pipelines

```
pipeline {
  agent any

  stages {
    stage('doe') {
```

```
    steps{
      sh 'ls -alh'
    }
  }
}
```

Jenkins

Dit is een open-source automatiseringstool voor CI/CD

- Doel
 - Automatiseren van taken zoals builds, testen en deploys
- Kenmerken:
 - Plug-in ondersteuning voor Git, docker, kubernetes
 - Configureerbaar via GUI pipelines
 - Kan draaien op elke platform

Vendor Agnostic

- kubernetes is vendor agnostic omdat het kan draaien op elke cloud
- Voordelen
 - Meer flexibiliteit
 - Eenvoudiger om te switchen tussen providers (bijv. AWS, Azure)
 - Minder risico op vendor lock-in.

Vendor specific

Een vendor specific oplossing is gebonden aan de technologie van een specifieke leverancier. Het biedt vaak geoptimaliseerde integratie en unieke functies, maar kan leiden tot vendor lock-in (moeilijk overstappen naar een andere leverancier)

Een goed voorbeeld hiervan is

- AWS lambda is vendor specific omdat het alleen binnen de AWS-cloud werkt

Verschillende testen:

Voorbeelden testen

- NUnit → Unit testen
- Postman → Integratie testen
- SeleniumIDE → Regression testen
- Newman → Documentatie testen
- OWASP ZAP → Security testen
- Katalon → Functional testen
- Webpagetest → system testen
- Gatling → performance testen

Artefacten binnen een devops omgeving en op welke manier dit gebruikt kan worden

- **Consistentie tussen omgevingen**
 - Artifacts worden eenmalig gegenereerd (bijvoorbeeld tijdens een build) en blijven onveranderd tijdens het deploymentproces. Dit garandeert dat dezelfde versie van de code in alle omgevingen wordt gebruikt (development, testing, staging, en productie).
 - Voorkomt verschillen die kunnen ontstaan door het opnieuw bouwen van de code in verschillende omgevingen.
- **Reproduceerbaarheid**
 - Artifacts worden opgeslagen in een repository, zodat eerdere versies van de software altijd opnieuw kunnen worden ingezet of getest.
 - Dit helpt bij het debuggen en rollbacken van wijzigingen.
- **Efficiëntie**

- Het gebruik van vooraf gebouwde artifacts bespaart tijd doordat builds niet telkens opnieuw hoeven te worden uitgevoerd.
- Teams kunnen zich richten op testen en deployen in plaats van steeds opnieuw te bouwen.
- **Traceerbaarheid**
 - Artifacts kunnen worden gekoppeld aan specifieke commits, builds, of versies, waardoor je precies weet welke code in productie is.
 - Dit vergemakkelijkt audits en versiebeheer.
- **Samenwerking tussen teams**
 - Development, QA, en Operations gebruiken dezelfde artifacts. Dit voorkomt miscommunicatie of incompatibiliteit tussen teams.

Hoe kunnen artifacts worden gebruikt in een DevOps-omgeving?

1. Artifact Repositories

- Artifacts worden opgeslagen in **artifact repositories**, zoals:
 - **JFrog Artifactory**: Voor allerlei artifacttypen zoals Docker-images, Maven-, en NPM-pakketten.
 - **Nexus Repository**: Vergelijkbaar met Artifactory.
 - **AWS S3** of **Azure Artifacts**: Voor cloudopslag.
- Deze repositories maken het eenvoudig om artifacts te beheren, te delen en te distribueren.

2. Continuous Integration (CI) Pipelines

- Tijdens CI-processen worden artifacts gegenereerd als onderdeel van het buildproces.
- Bijvoorbeeld:
 - Een `.jar` of `.war` bestand wordt gegenereerd na het bouwen van een Java-applicatie.
 - Een Docker-image wordt gebouwd en opgeslagen.
- Tools zoals Jenkins, GitLab CI/CD, en Azure Pipelines kunnen deze artifacts genereren en opslaan.

3. Continuous Delivery (CD) Pipelines

- Artifacts worden gedeployed naar verschillende omgevingen (bijv. testing, staging, productie) zonder opnieuw te bouwen.
- Deploymenttools zoals **Ansible**, **Terraform**, en **Kubernetes** gebruiken deze artifacts als invoer.

4. Infrastructure as Code (IaC)

- Configuratiebestanden en infrastructuurscripts worden ook als artifacts behandeld.
- Bijvoorbeeld Terraform-statebestanden of Kubernetes-manifests worden opgeslagen en beheerd als artifacts.

5. Rollback en Disaster Recovery

- Bij problemen in productie kan een eerdere versie van een artifact direct opnieuw worden ingezet.
- Artifacts maken het rollbackproces snel en betrouwbaar.

6. Containerization

- Docker-images, een veelvoorkomend type artifact, worden gebruikt om applicaties samen met hun afhankelijkheden in te pakken. Deze kunnen vervolgens consistent worden ingezet op verschillende omgevingen.

Waarom wordt er gewerkt met verschillende omgevingen en hoe ze verschillen:

Waarom werken met verschillende omgevingen?

- Veiligheid
 - Door ontwikkel- en testwerkzaamheden te scheiden van productie, voorkom je dat fouten of wijzigingen direct impact hebben op eindgebruikers
- Kwaliteitscontrole
 - Verschillende omgevingen zorgen ervoor dat nieuwe functies en fixes worden getest en gevalideerd voordat ze live gaan
- Teamwerk

- Verschillende teams (developers, QA, operations) kunnen onafhankelijk werken aan specifieke delen van de applicatie zonder elkaar te hinderen.
- **Reproduceerbaarheid**
 - Door code in meerdere stappen te testen (bijvoorbeeld lokaal, op staging, en in productie), kun je problemen vroeg detecteren en oplossen.
- **Risicobeheer**
 - Nieuwe wijzigingen worden eerst getest in een gecontroleerde omgeving voordat ze naar een omgeving met echte gebruikers worden gebracht.

De verschillende soorten omgevingen

- **Development (DEV)**
 - **Doel:** Hier schrijven en testen ontwikkelaars de code lokaal of op een gedeelde omgeving.
 - **Kenmerken:**
 - Frequent wijzigen van code.
 - Debugging-tools actief.
 - Niet-stabiel, fouten zijn normaal.
 - Echte of gesimuleerde afhankelijkheden (zoals een mock-database).
- **Test / Quality Assurance (QA)**
 - **Doel:** Hier wordt code uitgebreid getest door QA-teams.
 - **Kenmerken:**
 - Focus op functionele, integratie-, en regressietests.
 - Data is meestal gesimuleerd of anoniem gemaakt.
 - Kan geautomatiseerde testtools gebruiken.
 - Debugging is beperkt.
- **Staging / Pre-production**

- **Doel:** Een productie-achtige omgeving om te testen vóór release.
- **Kenmerken:**
 - Identiek aan productie (inclusief configuraties en afhankelijkheden).
 - Wordt gebruikt voor User Acceptance Testing (UAT).
 - Geen debugging of development tools actief.
 - Soms bevat het een subset van echte data.
- **Production (PROD)**
 - **Doel:** De live omgeving waar eindgebruikers toegang hebben tot de applicatie.
 - **Kenmerken:**
 - Stabiliteit en prestaties zijn cruciaal.
 - Echte gebruikersdata en workloads.
 - Monitoring- en logging-tools actief.
 - Geen fouten of instabiliteit acceptabel.

Voorbeeldscenario

Bij een webapplicatie:

1. Een ontwikkelaar bouwt en test nieuwe code in de **DEV**omgeving met behulp van mock-databases.
2. De nieuwe code wordt overgedragen aan de **QA**omgeving voor uitgebreide functionele en regressietests.
3. Vervolgens wordt de code in **STAGING** getest met productieachtige data en gebruikersscenario's.
4. Na goedkeuring wordt de code naar **PROD** gedeployed, waar deze live is voor echte gebruikers.

5. Flow & Feedback - pipelines

- Freestyle project
 - Freestyle is de traditionele manier om een project in jenkins op te zetten

- Het biedt een eenvoudige GUI om een buildproces te configureren
- Geschikt voor eenvoudige taken, zoals het uitvoeren van scripts, builds, of tests
- Kenmerken
 - **Configuratie:** Alles wordt via de Jenkins GUI ingesteld.
 - **Eenvoud:** Makkelijk te begrijpen en snel in te stellen, maar beperkt in mogelijkheden.
 - **Plugins:** Veel functionaliteiten vereisen extra plugins.
 - **Stateless:** Geen geavanceerde logica of gedeelde status tussen buildstappen.
 - **Beperkingen:** Moeilijk schaalbaar, lastig te hergebruiken voor complexe projecten of geautomatiseerde workflows.
- **Wat is een Pipeline Project?**
 - **Pipeline** is een nieuwere en krachtigere manier om CI/CD-processen te configureren en beheren.
 - Het gebruikt een **Jenkinsfile** (een tekstbestand in codevorm) om de volledige workflow te definiëren.
 - Ondersteunt complexe workflows en geavanceerde logica.

Kenmerken van Pipeline Projecten:

- **Configuratie:** Wordt geschreven als code in een declaratief of script-stijl formaat (Jenkinsfile).
- **Flexibiliteit:** Ondersteunt conditionele logica, loops, en dynamische workflows.
- **Versiebeheer:** Jenkinsfile kan samen met de broncode in versiebeheer worden opgeslagen.
- **Modulair:** Stappen (stages) kunnen worden hergebruikt en parallel worden uitgevoerd.
- **Staat behouden:** Ondersteunt het bewaren van de build-status (bijvoorbeeld bij storingen of herstarts).
- **Schaalbaarheid:** Uitstekend geschikt voor grotere, complexe projecten of microservices.

6. Pipelines continued - continuous integration

Concepten

- Credential manager
 - Een beveiligde opslaglocatie in Jenkins om gevoelige gegevens zoals wachtwoorden, API-tokens, SSH-sleutels en certificaten te beheren
 - Hoe werkt het
 - De beheerders voegen credentials toe aan Jenkins Credential Manager. Deze gegevens worden versleuteld opgeslagen.
 - Gebruiken: Credentials kunnen worden opgevraagd in Jenkins-pipelines via specifieke ID's, zonder dat de waarde in platte tekst wordt weergegeven.
 - Beveiliging: toegang tot credentials is beperkt tot geautoriseerde gebruikers of pipelines
- Maven
 - Een buildtool voor Java-projecten die helpt met het beheren van afhankelijkheden, het bouwen van software, en het uitvoeren van tests.
- UnitTest
 - Een type test dat individuele stukjes code (meestal functies of methoden) test om te controleren of ze correct werken
 - Focus op kleine, geïsoleerde onderdelen van de applicatie

```
@Test
public void testAddition() {
    assertEquals(5, Calculator.add(2, 3));
}
```

- JUnit
 - Een populair testframework voor Java waarmee je eenvoudig unit-test kunt schrijven en uitvoeren

Waarom gebruiken we de Global Tool configuration in Jenkins

Voordelen van de Global Tool Configuration

- Centrale Beheerbaarheid:
 - Alle tools worden op een plaats geconfigureerd, waardoor het eenvoudiger is om ze te beheren en te updaten.
- Consistentie:
 - Alle jobs gebruiken dezelfde versie van een tool, wat voorkomt dat verschillende versies van tools onverwachte buildproblemen veroorzaken.
- Schoolbaarheid:
 - Build-agents hoeven alleen de juiste tools geïnstalleerd te hebben, en Jenkins bepaalt automatisch welke versie moet worden gebruikt
- Flexibiliteit:
 - Verschillende tools en versies kunnen naast elkaar worden geconfigureerd, zodat verschillende projecten kunnen werken met specifieke versies.

Op welke manier worden unit-testen uitgevoerd en gerapporteerd

JUnit testen uitvoeren

- Java
 - In een Java applicatie worden JUnit tests meestal uitgevoerd met buildtools zoals Maven of Gradle. Jenkins kan deze tests uitvoeren en de resultaten verzamelen en rapporteren
- NodeJS-applicatie
 - Hoewel JUnit specifiek voor Java is, kun je in NodeJS vergelijkbare testframeworks gebruiken, zoals Jest, Mocha, of Jasmine. Deze frameworks genereren JUnit-compatibele rapporteren met plugins

7. DTAP & environments on demand

Concepten

- DTAP
 - DTAP staat voor Development, Test, acceptance, and production. Het is een gestructureerde aanpak om software door verschillende omgevingen te laten gaan
- Developments:
 - Hier schrijven ontwikkelaars code en testen ze nieuwe functies lokaal
 - Vaak gebruikt in combinatie met een versiebeheersysteem zoals Git
- Test
 - De software wordt in een gesimuleerde omgeving getest door QA-teams
 - Doel: Functionele en niet-functionele tests uitvoeren
- Acceptance:
 - Een pre-productieomgeving waarin eindgebruikers of klanten de software accepteren
 - Doel: Validatie van de software voordat deze live gaat
- Production:
 - De software draait in de live omgeving en is beschikbaar voor eindgebruikers
 - Strenge monitoring en beheer zijn essentieel
- Continuous deployment
 - Een geavanceerde DevOps-praktijk waarin nieuwe versies van software automatisch worden gedeployed naar productie zodra ze door alle tests heen komen
 - Verschilt van Continuous Delivery, waarbij goedkeuring nodig kan zijn
- Configuration Management
 - Beheert en automatiseert de configuratie van infrastructuur en software.

- Tools zoals Ansible, chef en puppet worden vaak gebruikt
- Zorgt ervoor dat omgevingen consistent en reproduceerbaar zijn.
- Applicatie Release Automation (ARA)
 - Automatisering van het proces om applicaties van ontwikkeling naar productie te brengen.
 - Vaak geïntegreerd met CI/CD-pipelines
 - Tools: Jenkins
- Docker
 - Een platform voor het bouwen, uitvoeren en beheren van containers.
 - Containers zijn lichte, geïsoleerde runtime-omgevingen die applicaties en hun afhankelijkheden bevatten
 - Zorgt voor consistentie tussen ontwikkel- test en productieomgevingen.
- Mutable vs. Immutable
 - Mutable infrastructure:
 - Infrastructuur die aangepast kan worden na creatie.
 - Voorbeeld: Updates toepassen op een draaiende server.
 - Immutable infrastructure:
 - Infrastructuur die niet meer verandert nadat deze is gedeployed.
 - Bij wijzigingen wordt een nieuwe versie uitgerold.
 - Populair in combinatie met tools zoals Docker en kubernetes

"Build once, Deploy anywhere" past binnen het DTAP model

Het concept **"Build Once, Deploy Anywhere"** past binnen het **DTAP-model** omdat het helpt om consistentie, betrouwbaarheid en efficiëntie te waarborgen bij het overbrengen van software van de ene omgeving naar de andere (bijvoorbeeld van Development naar Test, Acceptance en uiteindelijk Production). Hier is hoe dit principe in DTAP integreert:

Wat is "Build Once, Deploy Anywhere"?

- **"Build Once"**: De applicatie (inclusief alle bijbehorende afhankelijkheden) wordt één keer gebouwd in een gecontroleerde en consistente ontwikkelomgeving.
- **"Deploy Anywhere"**: Hetzelfde artefact (bijvoorbeeld een container-image, een gecompileerde binary of een pakket) wordt vervolgens naar elke omgeving in de DTAP-keten gedistribueerd, zonder dat het opnieuw wordt opgebouwd.

Voorbeelden van mutable and immutable infrastructuur

- Multable infrastructuur
 - Traditionele servers
 - Fysieke of virtuele server waarop je handmatig of via script wijzigingen aanbrengt.
 - Voorbeeld:
 - Een applicatieserver waarop een systeembeheerder een nieuwe versie van software installeert door een bestaand bestand te overschrijven.
 - Updates worden toegepast met tools zoals SSH en handmatige configuratie
 - Configuratiebeheer Tools:
 - Tools zoals Ansible, puppet, of Chef passen veranderingen toe op bestaande infrastructuur
 - Voorbeeld:
 - Een bestaand webservercluster wordt bijgewerkt door een configuratiescript dat bijvoorbeeld een nieuwe versie van NGINX installeert en de configuratie wijzigt
 - Database Servers
 - Bijwerken van een database rechtstreeks in een draaiende omgeving
 - Voorbeeld:

- Een databasebeheer voegt een nieuwe kolom toe aan een productiedatabase of voert direct een script uit
- voordelen
 - **Flexibiliteit:**
 - Je kunt snel kleine wijzigingen aanbrengen zonder een volledig nieuwe systeem te bouwen
 - Handig bij debugging of noodsituaties
 - **Lage initiële complexiteit:**
 - Je kunt bestaande infrastructuur aanpassen zonder complexe build-processen.
 - Minder tools of expertise nodig om te starten.
 - **Geschikt voor legacy-systemen:**
 - Veel bestaande applicaties en infrastructuren zijn ontworpen om updates direct toe te passen.
- Nadelen
 - **Configuratedrift:**
 - Over tijd kunnen verschillen ontstaan tussen systemen door handmatige of ongedocumenteerde wijzigingen.
 - Dit kan leiden tot inconsistenties en moeilijk te reproduceren bugs.
 - **Moeilijk te debuggen:**
 - Als een probleem optreedt, is het lastig te achterhalen welke wijzigingen dit hebben veroorzaakt.
 - **Onvoorspelbare staat:**
 - Omdat systemen aangepast worden, kunnen er afwijkingen ontstaan die in testomgevingen niet zichtbaar waren.
- Voorbeelden van Immutable Infrastructure
 - Voordelen:
 - **Consistentie:**

- Elke omgeving (Development, Test, Acceptance, Production) draait exact dezelfde versie van de infrastructuur en applicaties.
- Elimineert configuratiedrift.
- **Betrouwbaarheid:**
 - Omdat wijzigingen niet in-place plaatsvinden, wordt de kans op fouten door menselijke interactie verminderd.
- **Eenvoudig herstel:**
 - Bij problemen kun je snel teruggaan naar een eerdere, goedwerkende versie van de infrastructuur.
- **Schaalbaarheid:**
 - Immutable infrastructuur werkt goed met geautomatiseerde scaling-tools, zoals Kubernetes en auto-scaling groepen in AWS.
- **Compatibiliteit met moderne DevOps:**
 - Ideaal voor containerisatie en cloud-native omgevingen waar consistentie en automatisering belangrijk zijn.
- nadelen:
 - **Complexiteit in implementatie:**
 - Het vereist een goede CI/CD-pipeline en kennis van tools zoals Docker, Kubernetes, of Packer.
 - Teams moeten investeren in nieuwe processen en workflows.
 - **Tijdrovende updates:**
 - Omdat bij elke wijziging een nieuwe versie wordt uitgerold, kan dit langer duren dan direct een wijziging toepassen.
 - **Opslag en kosten:**
 - Omdat elke wijziging een nieuwe versie oplevert, kan de opslagruimte voor images, containers of builds toenemen.
 - Cloud-opslag en artifactrepositories kunnen duurder worden.
 - **Aanpassingen zijn minder flexibel:**

- Bij urgente problemen is het minder makkelijk om snel een kleine wijziging toe te passen, omdat je een volledig nieuwe versie moet bouwen.
- Docker containers
 - Containers worden gebouwd als een vast image (docker image) dat niet gewijzigd wordt nadat het gebouwd is
 - Voorbeeld:
 - Als een nieuwe versie van een applicatie nodig is, wordt een nieuwe container-image gebouwd en uitgerold. De oude container wordt verwijderd
- Golden Images:
 - Virtual Machine (VM) images die vooraf zijn geconfigureerd met een specifieke applicatieversie en systeemconfiguratie
 - Voorbeeld:
 - Bij een update wordt een nieuw image gemaakt en uitgerold via een tool zoals Packer, terwijl de oude VM's worden vernietigd
- Kubernetes Pods:
 - In Kubernetes worden applicaties verpakt in pods. Bij een update wordt een nieuwe pod met een nieuwe containerimage gestart, terwijl de oude wordt beëindigd
 - Voorbeeld
 - een deployment in kubernetes gebruikt Rolling Updates om nieuwe containers uit te rollen zonder de bestaande containers aan te passen

8.Flow & feedback: Integrated testing

Concepten

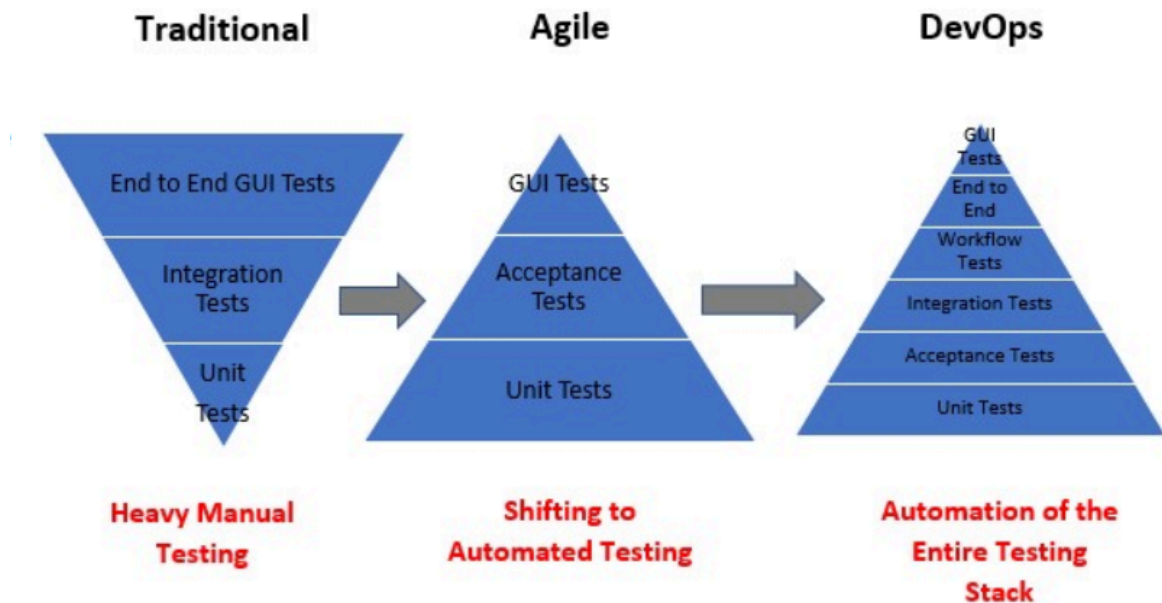
- Functional Testing
 - Wat is het?

- Testen of de applicatie werkt volgens de functionele eisen en specificaties
 - Controleert of elke functie het verwachte resultaat levert
- Voorbeeld:
 - Een inlogsysteem testen: wanneer een gebruiker het juiste wachtwoord invoert, wordt toegang verleend.
- Tools:
 - Selenium, TestComplete
- Non-Functional Testing
 - Wat is het?
 - Testen van aspecten die niet direct verband houden met specifieke functies, zoals prestaties, gebruiksvriendelijkheid, beveiliging, en schaalbaarheid.
 - Focus ligt op "hoe" het systeem werkt
 - Voorbeeld:
 - Testen hoe snel een pagina laadt bij 100 gelijktijdige gebruikers
 - Tools:
 - JMeter, LoadRunner
- Integration Testing
 - Wat is het?
 - Testen van de interactie tussen verschillende modules of systemen om te verifiëren dat ze goed samenwerken
 - Controleert gegevensuitwisseling en afhankelijkheden tussen componenten
 - Voorbeeld:
 - Het testen van een webshop: controleren of het voorradbeheer en de betalingsmodule goed samenwerken.
 - Tools:
 - Postman, soapUI

- Regression Testing
 - Wat is het?
 - Verifiëren dat bestaande functionaliteit niet is aangetast door nieuwe wijzigingen of updates
 - Detecteert bugs die door nieuwe code zijn geïntroduceerd
 - Voorbeeld:
 - Na een update aan een zoekfunctie controleren of andere delen van de applicatie (zoals filters) nog steeds correct werken.
 - Tools:
 - Selenium, QTP
- Performance Testing
 - Wat is het?
 - Testen van de snelheid, responsiviteit en stabiliteit van een systeem onder normale en piekbelasting.
 - Onderdeel van non-functional testing
 - Voorbeeld:
 - Meten hoe snel een pagina laadt bij een gemiddelde gebruiker.
 - Tools:
 - JMeter, gatling
- Security Testing
 - **Wat is het?**
 - Testen van de beveiliging van een applicatie tegen aanvallen, gegevensdiefstal en andere bedreigingen.
 - **Voorbeeld:**
 - Controleren of gebruikers zonder juiste rechten toegang kunnen krijgen tot gevoelige informatie.
 - **Tools:**
 - OWASP ZAP, Burp Suite.
- Load Testing

- Wat is het?
 - Testing hoe het systeem presteert onder een verwachte werklast.
 - Controleert of het systeem betrouwbaar is en voldoet aan de prestatievereisten.
- Voorbeeld:
 - Simuleren van 1000 gebruikers die tegelijkertijd een online winkel bezoeken
- Tools:
 - Apache JMeter, BlazeMeter
- Stress Testing
 - **Wat is het?**
 - Testen hoe een systeem zich gedraagt onder extreme of buitengewone belasting, vaak boven de normale capaciteitslimieten.
 - Doel: Achterhalen hoe en wanneer een systeem faalt.
 - **Voorbeeld:**
 - Testen hoe een e-commercewebsite reageert als 10.000 gebruikers tegelijkertijd proberen in te loggen.
 - **Tools:**
 - Locust, LoadRunner.

De test pyramide reproduceren en bij elke laag concreet



Laag	Focus	Kenmerken	Tools
Unit Tests	Kleine code-eenheden in isolatie	Snel, goedkoop, direct feedback.	pytest, JUnit, Jest
Integration Tests	Samenwerking tussen modules	Langzamer dan unit-tests, gericht op interfaces.	Postman, SoapUI
End-to-End Tests	Volledige workflows testen	Traag, duur, complex. Test de gebruikerservaring.	Selenium, Cypress

Waarom is het non-ideal testing pyramid geen goede use case

De non-ideal testing pyramid, wordt beschouwd als een slechte aanpak omdat de balans tussen de soorten tests niet goed is.

1. Te veel afhankelijkheid van End-to-end Tests

- Langzaam en kostbaar
 - End-to-end-tests nemen veel tijd in beslag omdat ze volledige workflows testen, afhankelijk zijn van externe systemen (zoals databases en API's) en vaak een gecompliceerde setup vereisen.
 - Dit verhoogt de feedbacktijd voor ontwikkelaars, waardoor iteraties langzamer worden.
- Fragiel:

- Deze tests zijn gevoelig voor veranderingen in de applicatie of infrastructuur.
- Zelfs kleine wijzigingen in de UI of backend kunnen leiden tot een groot aantal mislukte tests.
- Moeilijk te onderhouden:
 - Als er veel end-to-end-tests zijn, kost het veel moeite om ze te onderhouden bij elke codewijziging.

2. Gebrek aan Unit Test

- **Lange debugtijd:**
 - Als een fout optreedt, is het moeilijk om precies te achterhalen waar het probleem ligt zonder voldoende unit-tests.
 - Unit-tests helpen problemen vroeg in het ontwikkelproces op te sporen en te isoleren.
- **Hogere kosten bij fouterstel:**
 - Fouten die pas worden ontdekt in end-to-end-tests zijn duurder om op te lossen dan fouten die worden gevonden in unit-tests.

3. Onduidelijke testcoverage

- **Risico op blinde vlekken**
 - Met te weinig unit-tests en integratietests kunnen ontwikkelaars over het hoofd zien hoe specifieke functies of modules individueel werken.
 - Hierdoor zijn er minder garanties dat de applicatie betrouwbaar is.

4. Inefficiënt gebruik van middelen

- **Overbelasting van CI/CD-pijplijnen:**
 - End-to-end-tests vereisen vaak volledig geconfigureerde testomgevingen, wat resource-intensief is.
 - Dit kan de CI/CD-pijplijn vertragen en de feedbackcyclus frustreren.

9. Feedback-monitoring & reporting

concepten

Monitoring

- Wat is het?
 - Het continu observeren van systemen, applicaties en infrastructuur om prestaties beschikbaarheid en functionaliteit te controleren
- Doel:
 - Problemen opsporen en oplossen voordat ze gebruikers beïnvloeden
- Voorbeeld:
 - Het monitoren van server CPU gebruik en waarschuwingen genereren bij hoge belasting

Metrics

- Wat is het?
 - Kwantitatieve gegevens die inzicht geven in de prestaties van systemen of applicaties.
- Doel: Helpen bij het meten en beoordelen van de gezondheid van een systeem.

Log Aggregatie

- Wat is het?
 - Het verzamelen, samenvoegen en analyseren van logbestanden uit verschillende bronnen
- Doel:
 - Patronen, fouten en trends identificeren in applicatie- en systeemlogs
- Voorbeeld:
 - Alle server-logs centraliseren in een platform zoals ELK Stack (Elasticsearch, Logstash, Kibana) om eenvoudig foutopsporing uit te voeren.

SNMP (Simple Network Management Protocol)

- Wat is het?
 - Een protocol voor het verzamelen van informatie en beheren van netwerkkapparaten zoals routers, switches en servers
- Doel
 - Netwerkkapparatuur bewaken en beheren
- Voorbeeld:
 - Het ophalen van bandbreedtegebruik van een router via SNMP.

MIB (Management Information Base)

- Wat is het?
 - Een database die wordt gebruikt door SNMP om informatie over netwerkkapparaten te structureren en toegankelijk te maken.
- Voorbeeld:
 - Een MIB kan objecten definiëren zoals CPU gebruik of bandbreedtegebruik die door SNMP kunnen worden opgevraagd

Agent Software

- Wat is het?
 - Software die op een systeem draait en gegevens verzamelt over de prestaties, status of logs en deze naar een monitoringplatform stuurt
- Voorbeeld:
 - De datadog-agent verzamelt gegevens van een server en stuurt deze naar het Datadog dashboard

Application Performance Management (APM)

- Wat is het?
 - Het monitoren en beheren van de prestaties en beschikbaarheid van softwareapplicaties.
- Doel:
 - Problemen in applicaties identificeren, zoals vertragingen, foutcode of crashes
- Voorbeeld:

- Traceren van hoe lang een specifieke databasequery duurt in een applicatie

Proactive Monitoring

- Wat is het?
 - Monitoring waarbij proactief wordt gezocht naar tekenen van problemen voordat ze impact hebben
- Voorbeeld:
 - Een waarschuwing instellen voor hoge CPU-gebruik of lage diskruimte voordat deze een kritieke grens bereikt

Predictive monitoring

- Wat is het?
 - Het gebruik van data-analyse en machine learning om toekomstige problemen of trends te voorspellen
- Voorbeeld:
 - Voorspellen dat een server binnenkort overbelast raakt door een toenemende trend in gebruikersactiviteit

AIOps (Artificial Intelligence for IT Operations)

- Wat is het?
 - Het gebruik van AI en machine learning om IT-operaties te verbeteren, zoals het automatisch detecteren en oplossen van problemen.
- Voorbeeld:
 - Een AIOps-tool identificeert afwijkende netwerkactiviteit en schakelt automatisch bepaalde servers uit om aanvallen te voorkomen

Low-Risk Release

- Wat is het?
 - Release-methoden die ontworpen zijn om de impact van problemen bij nieuwe releases te minimaliseren.
- Voorbeelden:
 - Rollback-mogelijkheden, uitgebreide tests vooraf

Rolling Deploys

- Wat is het?
 - Release-strategie waarbij updates geleidelijk worden uitgerold naar een deel van de servers, terwijl de rest op de oude versie blijft draaien
- Doel:
 - Het minimaliseren van downtime en impact
- Voorbeeld:
 - Het updaten van een cluster server een voor een, zodat gebruikers nooit volledig zonder service zitten

Blue/Green Deploys

- Wat is het?
 - Een strategie waarbij 2 identieke productieomgeving (blue en green) worden gebruikt. De nieuwe versie wordt uitgerold naar de inactieve omgeving, waarna verkeer wordt omgeschakeld.
- Doel:
 - Veilig updates implementeren zonder downtime.
- Voorbeeld:
 - De "blue"-omgeving draait live; de update wordt uigerold naar "green". Na testen wordt al het verkeer naar "green" omgeleid

canary Deploys

- Wat is het?
 - Een release-strategie waarbij een nieuwe versie wordt uigerold naar een klein percentage van gebruikers, voordat deze breder wordt uitgerold.
- Doel:
 - Problemen opsporen in een beperkte groep gebruikers.
- Voorbeeld:
 - Slechts 5% van de gebruikers krijgt een nieuwe versie van een app. als er geen problemen optreden, wordt deze uitgerold naar de rest.

A/B Testing

- Wat is het?
 - Een techniek om 2 varianten van een functionaliteit of design te testen door verschillende gebruikersgroepen elk een variant te laten ervaren.

Kan voorbeelden van welke soort data er allemaal gelogd kan worden

- Systeemlogs
 - **Serverinformatie:** Uptime, belasting, geheugen- en CPU-gebruik.
 - **Fouten en waarschuwingen:** Foutcodes, stacktraces, crashes.
 - **Beveiligingsgebeurtenissen:** Mislukte inlogpogingen, wijziging van rechten.
 - **Updates en wijzigingen:** Installatie van nieuwe software of updates.
- Toepassingslogs
 - **Gebruikersactiviteiten:** Inlog- en uitlogtijden, bekeken pagina's, aangeklikte knoppen.
 - **Transacties:** Bestellingen, betalingen, retouren.
 - **Foutopsporing:** Debugging logs, API-verzoeken en -antwoorden.
 - **Prestatiegegevens:** Reactietijden van functies of onderdelen.
- Beveiligingslogs:
 - **Authenticatiegegevens:** Wie is ingelogd, vanaf welk IP-adres, met welke rol.
 - **Firewallverkeer:** Verkeersregels die zijn toegepast of geblokkeerde verzoeken.
 - **Incidentregistratie:** Detectie van malware, verdacht gedrag.
 - **Toegang tot gevoelige bestanden:** Wie heeft wat geopend, bewerkt, of verwijderd.
- Netwerklogs
 - **Verkeersinformatie:** Bron- en doel-IP-adressen, gebruikte poorten.
 - **Protocolinformatie:** HTTP, HTTPS, FTP, etc.

- **Pakketinformatie:** Analyse van inkomende en uitgaande pakketten.
- **Verbindingsstatus:** Open, gesloten, verbroken verbindingen
- Gebruikersgedraglogs
 - **Clicks en scrolls:** Welke elementen werden interactief gebruikt.
 - **Sessies:** Tijd die doorgebracht is op een website/app, paden van gebruikers.
 - **Aankoopgedrag:** Producten bekeken, aan winkelmandje toegevoegd.
 - **Feedback:** Ingevulde formulieren, beoordelingen.
- Hardware-/IoT-logs
 - **Statusgegevens:** Temperatuur, stroomverbruik, batterijstatus.
 - **Sensorwaarden:** Gegevens van sensoren zoals temperatuur, druk, locatie.
 - **Verbindingslogs:** Communicatie met andere apparaten.
- Operationele logs
 - **Batchverwerking:** Tijdstippen, status (voltooid of mislukt).
 - **Jobs en taken:** Planning en uitvoering.
 - **Automatisering:** Resultaten van scripts of pipelines.
- Analyse- en marketinglogs
 - **Advertentieprestaties:** Klikken, vertoningen, conversies.
 - **SEO-gegevens:** Zoekwoorden, bounce rates.
 - **A/B-testresultaten:** Welke variant presteert beter.

Uitleg van het SNMP protocol en waar die voor gebruikt wordt:

SNMP (Simple Network Management Protocol) is een protocol dat wordt gebruikt voor het beheren en monitoren van netwerkkapparaten, zoals routers, switches, servers, printers en andere apparaten die met het netwerk verbonden zijn. Het biedt een gestandaardiseerde manier om informatie over apparaten op te vragen en instellingen te wijzigen

Onderscheid maken tussen classieke en moderne deployment strategien

Low Risk Releases - Bringing it together

Classic Deploys

- Pro:
 - Simple, fast, cheap
- Cons:
 - Risk, outages, slow rollback, unemployment

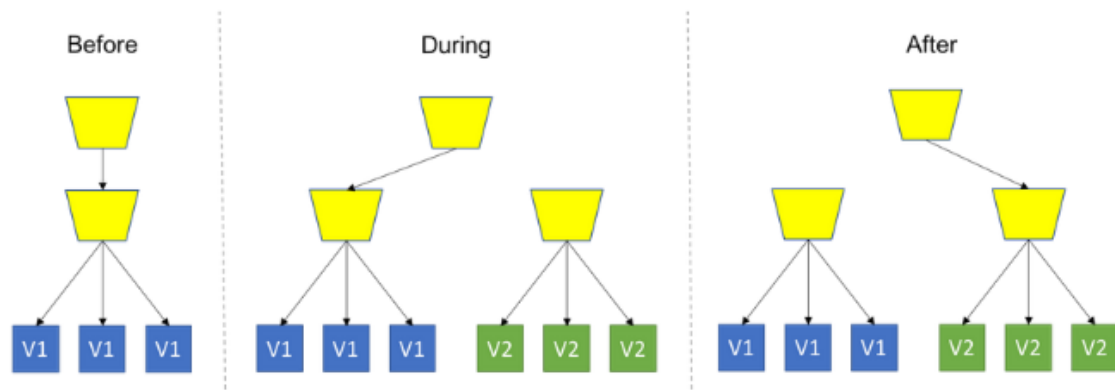
Rolling deploys

- Pros:
 - Simple, cheap, relatively simple to rollback, less risk than basic deployment.
- Cons:
 - App/DB needs to support both new and old artifacts.
 - Manual checks/verification at each increment could take a long time

Blue/Green Deploys

- Pros:
 - Simple, fast, well understood.
 - Less risk relative to other deployment strategies
 - Rapid rollback
- Cons:
 - Complex and expensive
 - Coverage may not identify all anomalies

Blue/Green deploys

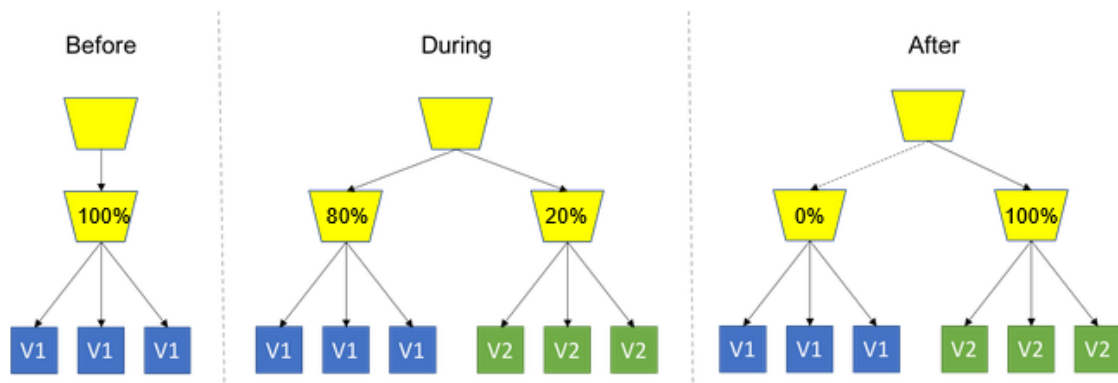


Canary deploys

Pros:

- Deploy in small phases
- Test in production with real users & use cases
- Cheaper than blue/green, Fast and safe rollback
- Cons:
 - Scripting canary deployments can be complex
 - required monitoring & instrumentation (APM, Log, Infra, End user, ...) for testing in production

Canary deploys



A/B Testing

- Pros:

- Fast, easy, and cheap way to test new features in production. Lots of tools exist to enable this.
- Cons
 - Experiments can sometimes break the app/service/user experience
 - Scripting AB tests can be complex
 - Database compatibility (schema changes, backward compatibility)

10. Integratieopdracht

YAML (yet another Markup language)

- Dit is een eenvoudige, leesbare opmaaktaal die vaak wordt gebruikt om configuratiebestanden te schrijven.
- Het wordt veel gebruikt in Devops-tool zoals Kubernetes, docker
- kenmerken:
 - Leesbaarheid:
 - YAML gebruikt inspringingen in plaats van ingewikkelde tags of haakjes
 - Datastructuren:
 - Onderstreunt lijsten, mappen, en scalaires

Webhooks

- Webhooks zijn een manier voor applicaties om in real-time meldingen te verzenden naar een andere systeem of applicatie via http-verzoeken

Environment variables

- Environment variables zijn sleutel-waardeparen die instellingen en configuratie-informatie bevatten en door een applicatie kunnen worden gelezen